

Parallelization

Lecture 19

March 25, 2026

Parallelization Steps

1. *Decomposition* of computation into tasks

- Identifying portions of the work that can be performed concurrently

2. *Assignment* of tasks to processes

- Assigning concurrent pieces of work onto multiple processes running in parallel

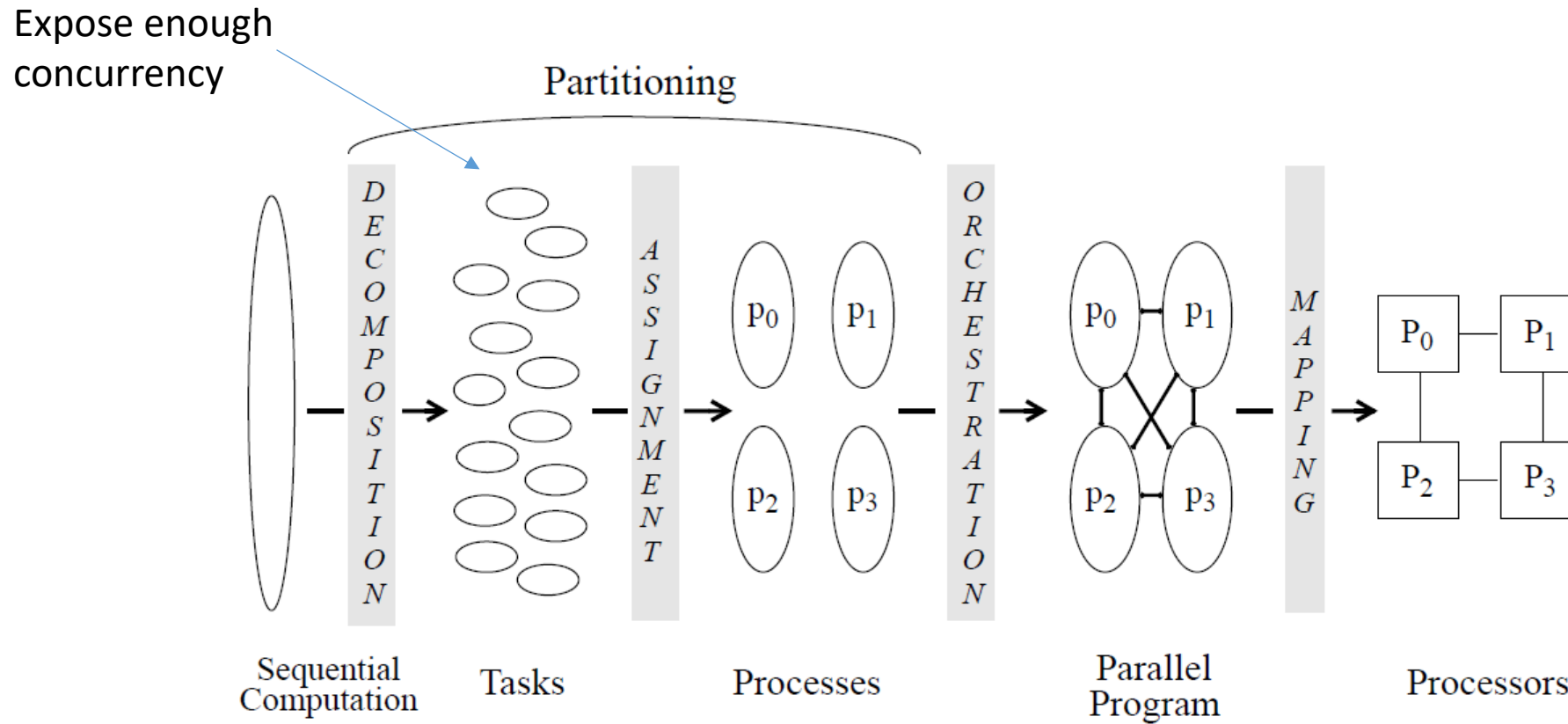
3. *Orchestration* of data access, communication and synchronization among processes

- Distributing the data associated with the program
- Managing access to data shared by multiple processes
- Synchronizing at various stages of the parallel program execution

4. *Mapping* of processes to processors

- Placement of processes in the physical processor topology

Illustration of Parallelization Steps

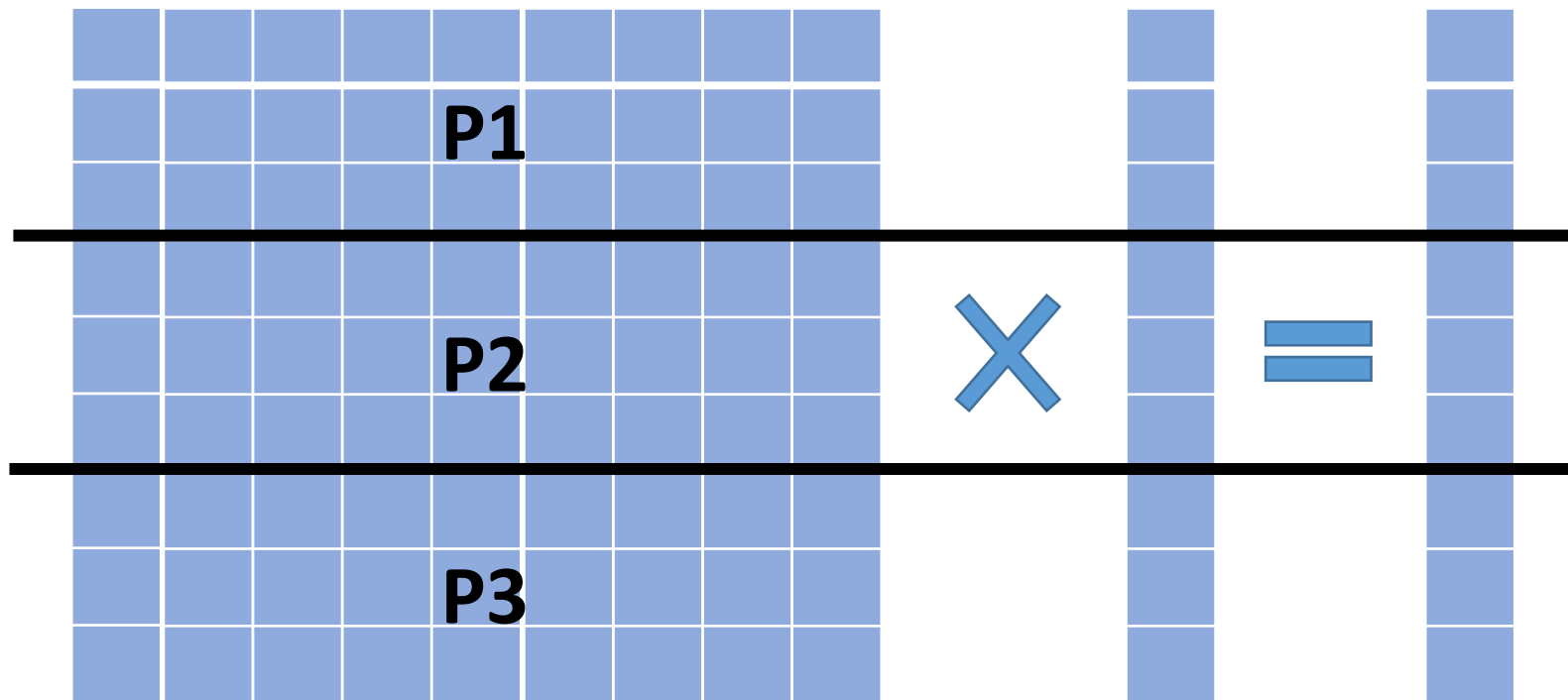


Source: Parallel Computer Architecture, Culler et al.

Performance Goals

- Expose concurrency
- Reduce inter-process communications
- Load-balance
- Reduce synchronization
- Reduce idling
- Reduce management overhead
- Preserve data locality
- Exploit network topology

Matrix Vector Multiplication – Decomposition



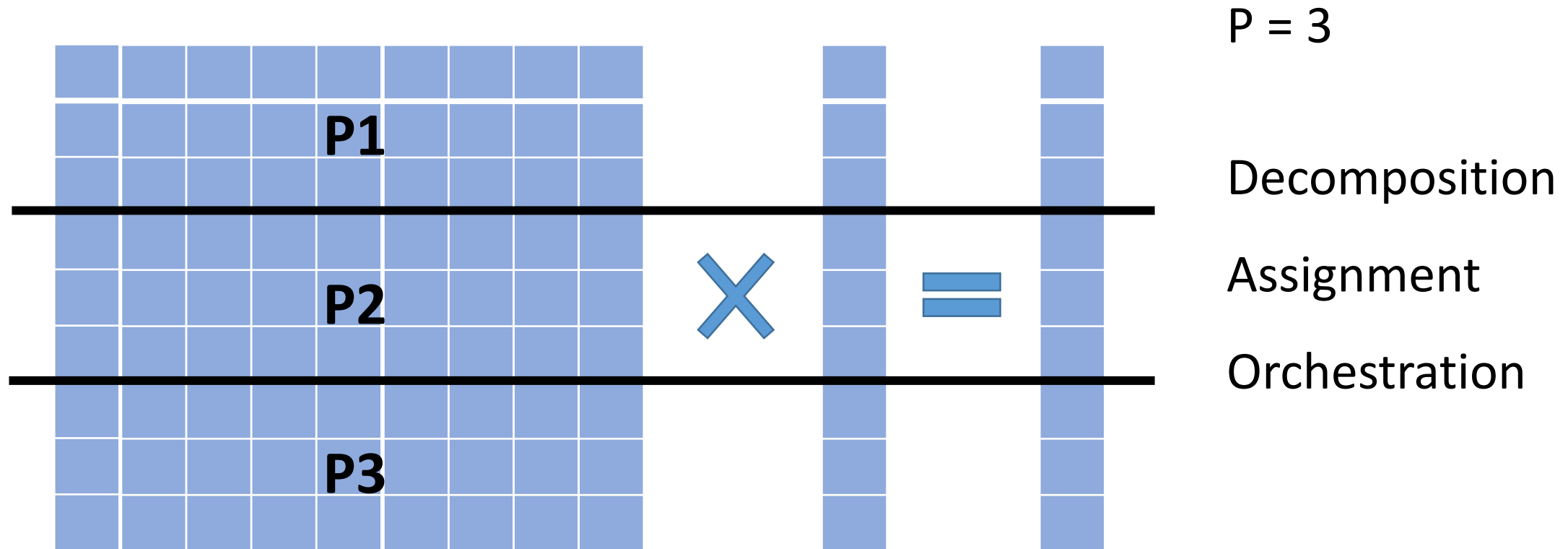
$P = 3$

Decomposition

Identifying portions of the work that can be performed concurrently

Assignment

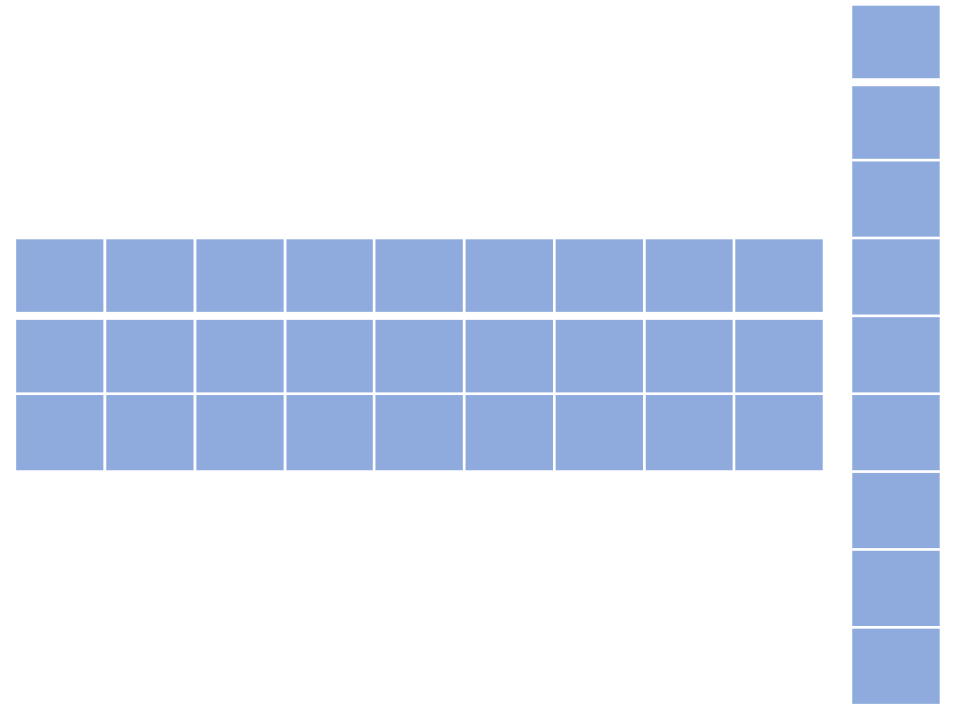
Matrix Vector Multiplication – Orchestration



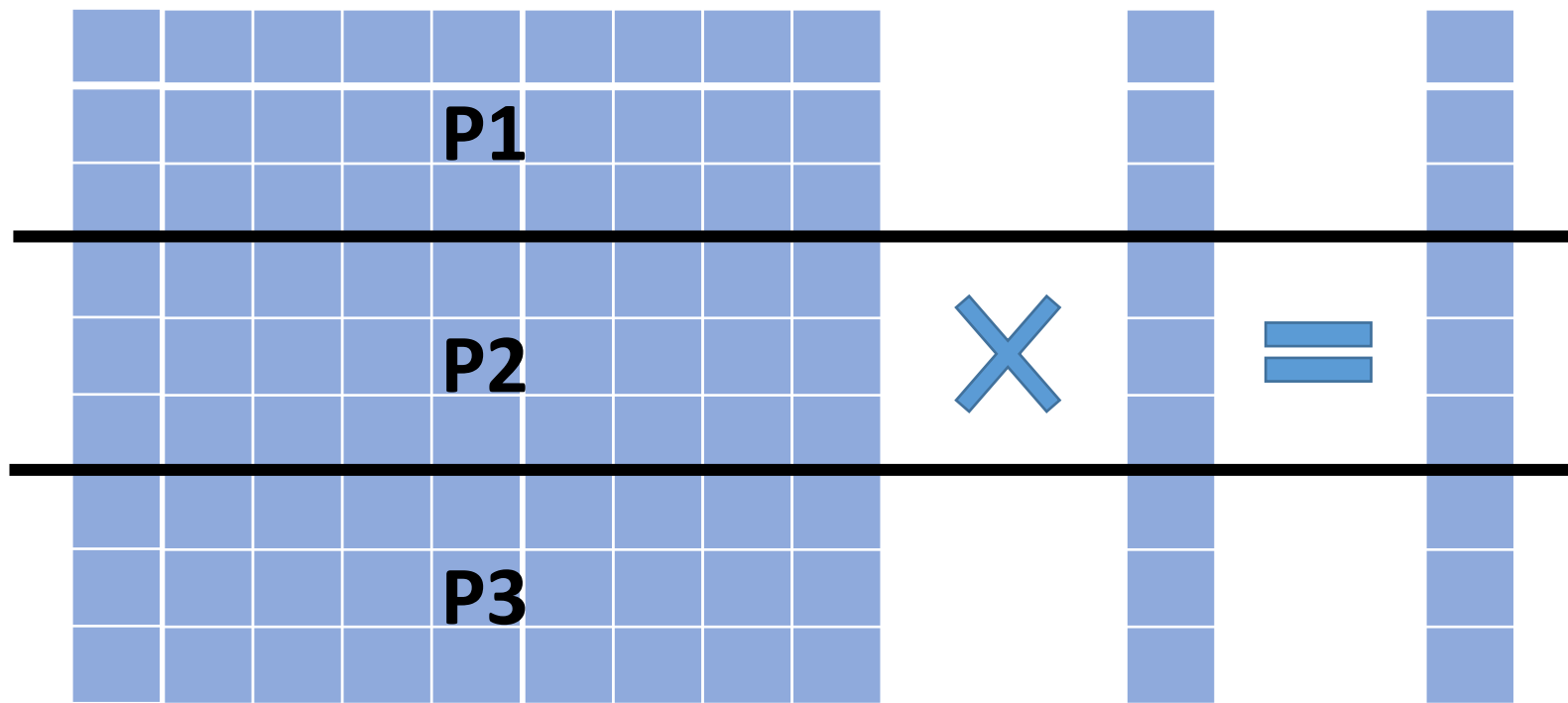
- Initial communication
 - Distribute (read by process 0) or parallel reads
- Final communication

Write the Code (Homework)

- Obtaining the vector (length N)
 - MPI_Bcast (in case the entire vector was read by rank 0)
 - MPI_Allgather
- Obtaining the matrix (read by rank 0)
 - MPI_Scatter
- Computation: subMatrix (RXC) X vector (CX1)
- Final communication
 - MPI_Gather (result in one rank)



Matrix Vector Multiplication – Orchestration



$P = 3$ (think of $P = 16384$ while devising solutions)

Decomposition

Assignment

Orchestration

- MPI_Allgather/
MPI_Bcast
- MPI_Scatter
- MPI_Gather

- Initial communication
 - Distribute (read by process 0) or parallel reads
- Final communication

Distribute using MPI_Bcast vs. MPI_Allgather

The actual comparison between the two approaches (read by 0 and parallel reads) should involve the read times as well

```
MPI_Init( &argc, &argv );  
MPI_Comm_rank( MPI_COMM_WORLD, &myrank );  
MPI_Comm_size( MPI_COMM_WORLD, &commsize );
```

Approach 1: Serial read by rank 0, MPI_Bcast

Approach 2: Parallel reads, MPI_Allgather

```
time = MPI_Wtime();  
MPI_Bcast (buf, N, MPI_FLOAT, 0, MPI_COMM_WORLD);  
etime = MPI_Wtime() - time;  
MPI_Reduce (&etime, &maxtime, 1, MPI_DOUBLE, MPI_MAX, 0, MPI_COMM_WORLD);  
if (!myrank) printf ("Time to bcast: %11.3lf\n", maxtime);
```

```
int count = N/commsize;  
time = MPI_Wtime();  
MPI_Allgather (buf, count, MPI_FLOAT, recvbuf, count, MPI_FLOAT, MPI_COMM_WORLD);  
etime = MPI_Wtime() - time;  
MPI_Reduce (&etime, &maxtime, 1, MPI_DOUBLE, MPI_MAX, 0, MPI_COMM_WORLD);  
if (!myrank) printf ("Time to allgather: %7.3lf\n", maxtime);
```

```
class for i in `seq 1 3` ; do mpirun -np 10 -hosts csews2,csews5,csews20 ./bcast-allgather 10000; echo ; done
Time to bcast:      0.014
Time to allgather:  0.021

Time to bcast:      0.018
Time to allgather:  0.009

Time to bcast:      0.012
Time to allgather:  0.007

class for i in `seq 1 3` ; do mpirun -np 10 -hosts csews2,csews5,csews20 ./bcast-allgather 100000; echo ; done
Time to bcast:      0.034
Time to allgather:  0.011

Time to bcast:      0.027
Time to allgather:  0.023

Time to bcast:      0.026
Time to allgather:  0.011

class for i in `seq 1 3` ; do mpirun -np 10 -hosts csews2,csews5,csews20 ./bcast-allgather 1000000; echo ; done
Time to bcast:      0.187
Time to allgather:  0.347

Time to bcast:      0.176
Time to allgather:  0.111

Time to bcast:      0.155
Time to allgather:  0.112
```

```
class for i in `seq 1 3` ; do mpirun -np 10 -hosts csews2,csews5,csews20 ./bcast-allgather 1000000; echo ; done
Time to bcast:      0.187
Time to allgather:  0.347

Time to bcast:      0.176
Time to allgather:  0.111

Time to bcast:      0.155
Time to allgather:  0.112

class for i in `seq 1 3` ; do mpirun -np 10 -hosts csews2,csews5,csews20 ./bcast-allgather 10000000; echo ; done
Time to bcast:      1.421
Time to allgather:  1.121

Time to bcast:      1.618
Time to allgather:  1.282

Time to bcast:      1.674
Time to allgather:  1.583

class for i in `seq 1 3` ; do mpirun -np 10 -hosts csews2,csews5,csews20 ./bcast-allgather 100000000; echo ; done
Time to bcast:      18.061
Time to allgather:  15.616

Time to bcast:      23.447
Time to allgather:  17.005

Time to bcast:      16.875
Time to allgather:  11.085
```

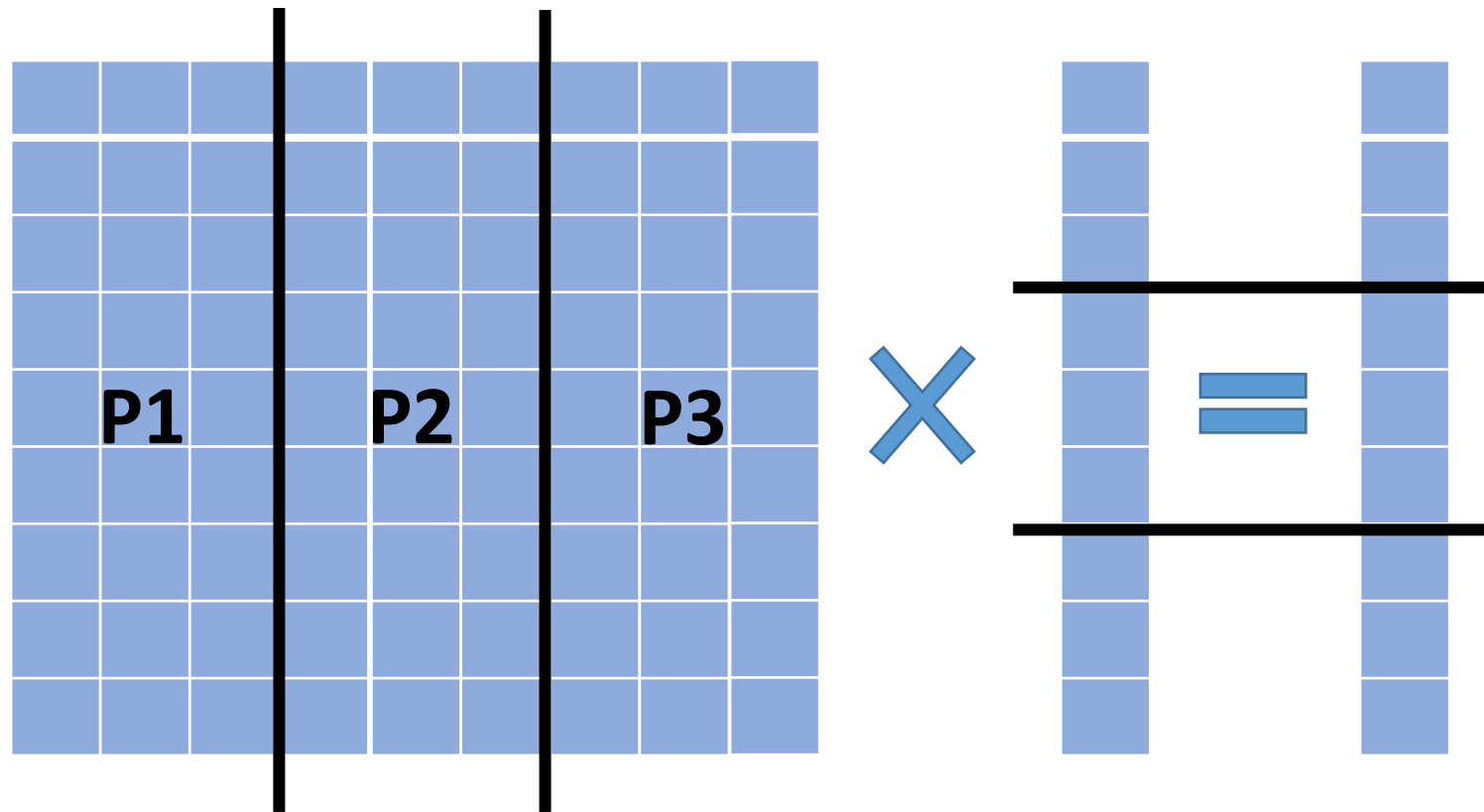
Broadcast Algorithms in MPICH

- Short messages
 - `< MPIR_CVAR_BCAST_SHORT_MSG_SIZE`
 - Binomial
- Medium messages
 - `MPI_Scatter + MPI_Allgather` (Recursive doubling)
- Large messages
 - `> MPIR_CVAR_BCAST_LONG_MSG_SIZE`
 - `MPI_Scatter + MPI_Allgather` (Ring)

`MPIR_CVAR_BCAST_SHORT_MSG_SIZE = 12288`

`MPIR_CVAR_BCAST_LONG_MSG_SIZE = 524288`

Matrix Vector Multiplication – Column-wise Matrix Decomposition



Decomposition
Assignment
Orchestration

- Initial distribution?
- Reduce

Row-wise vs. column-wise partitioning?

MPI_Bcast

10 rounds

0: 0.306482 1774363443.000000

MPI_Bcast

0: 0.059988 1774363443.000000

?

```
double sTime = MPI_Wtime();
```

```
for (int i=0; i<ROUNDS; i++)
```

```
    MPI_Bcast (buf, count, MPI_INT, 1, MPI_COMM_WORLD);
```

```
double eTime = MPI_Wtime();
```

Non-blocking Collectives

if I am rank 0

for (r=1; r<size; r++)

MPI_Send (data to r)

else

MPI_Recv (data from 0)

MPI_Bcast

if I am rank 0

for (r=1; r<size; r++)

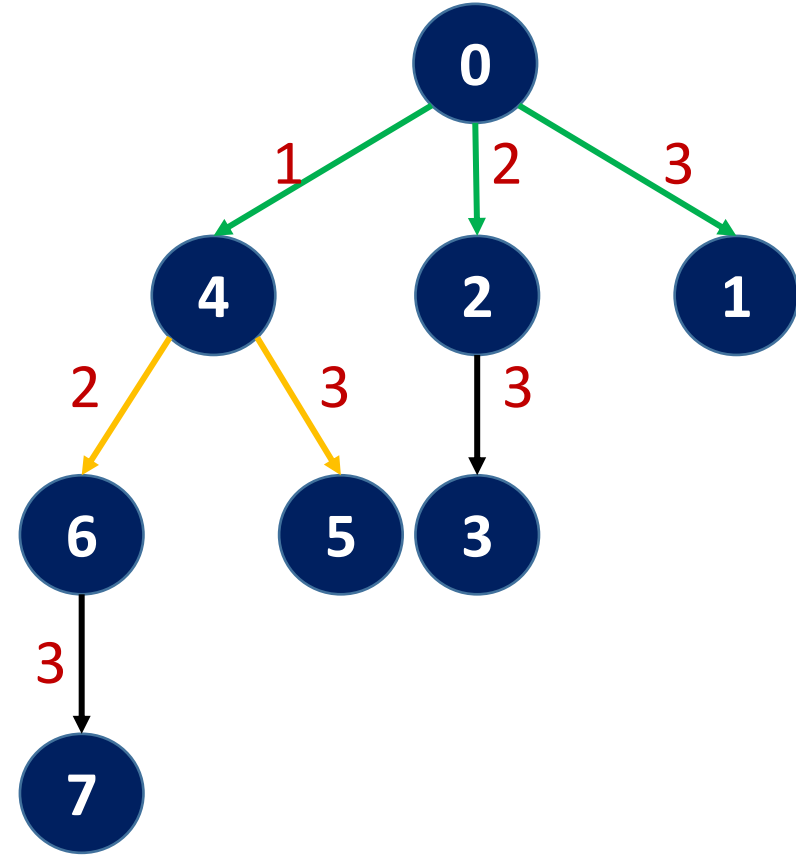
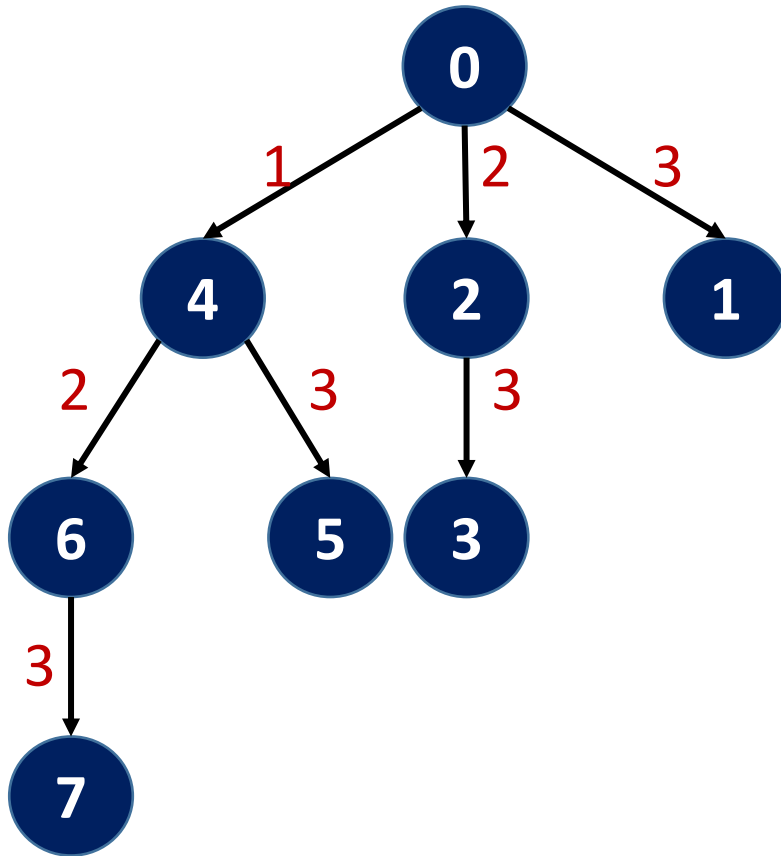
MPI_Send (data[r] to r)

else

MPI_Recv (data from 0)

MPI_Scatter

Blocking vs. Non-Blocking MPI_Bcast



Non-blocking Collectives

```
int MPI_Ibcast (  
    void *buffer,  
    int count,  
    MPI_Datatype datatype,  
    int root,  
    MPI_Comm comm,  
    MPI_Request *request  
)
```

```
int MPI_Igather (  
    const void *sendbuf,  
    int sendcount,  
    MPI_Datatype sendtype,  
    void *recvbuf,  
    int recvcount,  
    MPI_Datatype recvtype,  
    int root,  
    MPI_Comm comm,  
    MPI_Request *request  
)
```

Code

Anything missing?

Write a code that invokes MPI_Bcast
ROUNDS number of times

```
double sTime = MPI_Wtime();  
for (int i=0; i<ROUNDS; i++)  
    MPI_Bcast(buf, count,  
MPI_INT, 1, MPI_COMM_WORLD);  
double eTime = MPI_Wtime();
```

Write a code that invokes MPI_Ibcast
ROUNDS number of times

```
double sTime = MPI_Wtime();  
for (int i=0; i<ROUNDS; i++)  
    MPI_Ibcast(buf, count,  
MPI_INT, 1, MPI_COMM_WORLD,  
&request[i]);  
double eTime = MPI_Wtime();
```

Code

Incorrectly timed, why?

Communication may not have completed

```
double sTime = MPI_Wtime();
for (int i=0; i<ROUNDS; i++)
    MPI_Ibcast(buf, count,
MPI_INT, 1, MPI_COMM_WORLD,
&request[i]);
double eTime = MPI_Wtime();
MPI_Waitall (ROUNDS, request,
status);
```

Correctly timed

```
double sTime = MPI_Wtime();
for (int i=0; i<ROUNDS; i++)
    MPI_Ibcast(buf, count,
MPI_INT, 1, MPI_COMM_WORLD,
&request[i]);
MPI_Waitall (ROUNDS, request,
status);
double eTime = MPI_Wtime();
```

What's wrong?

```
mpirun -np 4 ./ibcast_time 10000000 1000
```

```
13.731223 2063425283
```

```
0.001899 0.000000 Incorrect data
```

```
10.720818 2063425283
```

MPI_Bcast

Incorrectly timed MPI_Ibcast

Correct MPI_Ibcast

```
mpirun -np 32 ./ibcast_time 1000000 1000
```

```
11.358705 19032026
```

```
0.038475 0.000000 ← Incorrect data  
(communication may  
not complete before  
MPI_Wait/MPI_Waitall  
is called)
```

MPI_Bcast

Incorrectly timed MPI_Ibcast

Correct MPI_Ibcast

MPI_Bcast

mpirun -np 20 ./exe 20000 200

7.08s

MPI_lbcast

mpirun -np 20 ./exe 20000 200

0.44s

MPI_Bcast

mpirun -np 20 ./exe 20000 2000

58.92s

MPI_lbcast

mpirun -np 20 ./exe 20000 2000

7.08s

mpirun -np 20 ./exe 1000 10

0.002718 584217146.000000

0.000290 584217146.000000

mpirun -np 20 ./exe 10000 10

0.005084 1083597142.000000

0.004471 1083597142.000000

mpirun -np 10 ./exe 200000 1000

MPI_Gather

12.805044 167760991.000000

MPI_lgather

6.162012 167760991.000000

More Timings for MPI_Gather/MPI_Igather

```
mpirun -np 60 -hosts csews1:15,csews2:15,csews4:15,csews10:15 ./exe  
200000 1000
```

```
624.085428 1433356434.000000
```

```
610.872046 1433356434.000000
```